Grounding and validation in language interfaces for symbolic robot task planning

Abstract

Language models can convert instructions into planning specifications, programs, or calls to robot skills. These outputs expose different opportunities for checking an instruction before and during execution. Existing reviews explain language grounding and the roles of foundation models in robotics, but a representation label alone does not identify which component checks object identity, action applicability, or task completion. This critical review examines interfaces between natural language and symbolic robot task planning through five questions: what is emitted, how objects are bound, where preconditions are represented, who validates the output, and how feedback changes subsequent decisions. A focused search and citation tracing connect classical planning foundations with robot systems and recent planning language benchmarks. The synthesis distinguishes goal and domain compilation, action and program generation, and skill selection, while treating monitoring and model repair as cross-cutting mechanisms. Across these mechanisms, stronger checks relocate uncertainty rather than remove it. A valid plan can satisfy an incorrectly translated goal; a well-typed skill call can reference the wrong object; and a learned feasibility score does not certify an action's preconditions. We develop an interface checklist, a worked counterexample, and a matched evaluation protocol that separates specification fidelity, symbolic validity, physical feasibility, and observed completion. These tools support explicit claims about interface reliability without assuming that a symbolic representation alone makes a robot dependable.

Keywords: robot task planning; language grounding; symbolic planning; large language models; PDDL; execution monitoring

1 Introduction

A robot asked to put the clean mug in a cupboard must resolve more than a sequence of verbs. It must identify a particular mug, determine whether its cleanliness is known, establish that the cupboard can be opened, and verify the resulting placement. These commitments become distributed across perception, language interpretation, task planning, and execution. Research on robots that use language has long treated the connection between linguistic meaning and sensorimotor representations as a central problem [1]. Large language models (LLMs) make it easier to propose interpretations and actions, but they do not settle where these commitments should be represented or checked.

This review studies the language to symbolic interface: the boundary at which an instruction becomes an explicit object, predicate, operator, program statement, or skill invocation that another component can consume. Symbolic does not mean that the entire robot uses a discrete world model. A Python call with an object identifier and a continuous pose has a symbolic interface even when perception and control remain learned. Conversely, a policy that maps an instruction and image directly to motor actions lies outside the main scope unless it exposes a task interface relevant to planning.

Several strong reviews already organize this area. Cohen and colleagues place robotic language grounding on a spectrum between symbols and embeddings [2]. Kim and colleagues cover communication, perception, planning, and control [3]. The manipulation survey by Yao and colleagues organizes how language supports state evaluation, policy conditioning, cognitive planning, and unified action models [4]. Broader agentic AI reviews discuss planners, tools, memory, and verification across application domains [5]. These works establish that representation, grounding, and feedback matter. The present review adopts a narrower question: when language is translated into a task interface, what evidence justifies accepting the resulting instruction for the next stage?

The contribution is a comparative synthesis of checking responsibilities, rather than a new planning algorithm or a claim to have introduced interface-based classification. In particular, recent work unifies planning language generation through intermediate representations [6]. Our complementary emphasis is the connection between those representations and robot objects, action applicability, and execution observations. A contemporary project-page survey also organizes robot control by action interfaces. The page listed its paper as forthcoming at access [7]. We therefore do not claim that using interfaces as an organizing principle is new.

Three questions guide the analysis. First, which information is fixed by the robot designer and which information is generated from language? Second, which errors can be rejected by a parser, symbolic model, feasibility estimator, or execution monitor? Third, how should results be reported so that a gain in one kind of validity is not mistaken for an improvement in another? Answering these questions gives researchers a basis for selecting and evaluating an interface without reducing the literature to a catalogue of LLM prompts.

2 Scope and evidence collection

2.1 Review design and search boundary

This is a focused critical review with a documented search trail, not an exhaustive systematic review or a meta-analysis. Searches were conducted on 19 October 2026 using web discovery queries and targeted searches of publisher and proceedings sites. Sources were then retrieved from arXiv, the Proceedings of Machine Learning Research, the ACL Anthology, IJCAI, robotics proceedings, journal sites, and author project pages. Backward citation tracing from surveys and primary papers supplied additional candidates. Queries combined robot task planning and language grounding with terms for the Planning Domain Definition Language (PDDL), action preconditions, object binding, programs, skills, scene graphs, validation, and feedback. Exact queries and source links are supplied in Online Resource 2.

The core scope covers language-conditioned task interfaces that emit formal specifications, explicit action sequences, programs, or skill invocations. The comparison emphasizes work from 2022 onward, alongside earlier planning and grounding foundations. Classical planning and task and motion planning provide background. Text-based embodied environments and domain-independent planning benchmarks are included when they isolate an interface mechanism that is relevant to robots. They are not treated as equivalent to physical robot evaluation. Pure motion control, general-purpose tool agents, and end-to-end vision-language-action policies are excluded from detailed comparison unless they clarify an interface boundary. English-language accessible sources are emphasized; this introduces language and access bias.

Selection was purposive and iterative. Papers were retained for an inspectable interface mechanism, a useful comparison, or an important limitation, rather than a reported success threshold. Duplicate preprint and proceedings versions were consolidated at the work level, with the published version preferred for bibliographic metadata. Public preprints lacking a verified publication record are explicitly labeled as preprints rather than presented as peer-reviewed results. No database-wide retrieval count, independent dual screening, registered protocol, or screening-flow diagram is claimed. This limits conclusions about completeness, prevalence, and the relative frequency of interface designs.

2.2 Extraction and synthesis

Targeted full-text reading covered the method descriptions and the evaluation or limitation passages needed for each substantive comparison. The evidence ledger records the source version, access status, relevant PDF pages or section, supported interpretation, and boundary of inference. Metadata were checked against publisher pages, official proceedings, and DOI registration records where available. Project documentation was used to clarify artifact availability and implementation interfaces, not to replace the paper's experimental evidence. Failed retrievals and metadata mismatches were retained in the research record rather than silently treated as successful verification.

We extracted five fields: output representation; object binding; the source of action preconditions or feasibility information; the authority and timing of checks; and the target of feedback or repair. These fields describe mechanisms, not an ordinal quality score. A method can combine several representations or move between them during execution.

Comparative tables therefore describe representative configurations in the cited papers. The full ledger is Online Resource 1; Online Resource 3 explains the coding rules and the proposed evaluation protocol. No experiments were rerun and no new robot performance data were generated.

The analysis distinguishes an observed result in a source study from an interpretation developed here. Reported mechanisms support the classification; claims that a particular architecture is universally better would require matched experiments beyond this review. Numerical performance values are not pooled because the robot platforms, low-level skills, prompts, world models, model versions, retry budgets, and success criteria differ. Table 1 locates this review relative to adjacent syntheses and recent methodological work.

Table 1 Relation to adjacent reviews and methodological syntheses

Source	Organizing emphasis	Contribution retained here	Additional question in this review
Cohen et al. [2]	Symbols and embeddings in robotic grounding	Representation has distinct benefits and costs	Which checker accepts the resulting task artifact?
Kim et al. [3]	Communication, perception, planning, control	Grounding spans the robotics stack	Who supplies object bindings and action conditions?
Yao et al. [4]	Roles of language in manipulation	Planning interacts with policies and perception	Which validity claim survives that interaction?
Service-robot review [8]	Input modalities in domestic task planning	Task planning depends on available modalities	What is checked after interpretation?
Kagitha et al. [6]	Intermediate representations for formalization	Representation and pipeline choices need comparison	How do formal artifacts connect to robot state and feedback?
This review	Binding, preconditions, validation, repair	A focused critical synthesis	Separate specification, symbolic, geometric, and observed outcomes

The dimensions are complementary. This table does not claim that prior work ignores grounding or validation.

3 An interface checklist for grounding and validation

3.1 Representation and responsibility are separate axes

PDDL expresses actions and planning problems through a formal vocabulary. Its temporal and numeric extensions support richer requirements than a simple ordered action list [9]. A

planner such as Fast Downward searches a specified symbolic problem [10]. Task and motion planning (TAMP) addresses the additional connection between discrete choices and continuous geometric constraints [11]. These foundations matter because an LLM may generate any part of the input to these components while leaving other parts fixed.

We describe an interface using five fields, summarized in Table 2. Representation specifies the emitted artifact. Binding connects linguistic references and symbolic arguments to scene entities. Preconditions describe when an action is applicable, or identify the alternative feasibility signal used. Validation identifies the component empowered to reject a proposal. Feedback specifies which observations become available and whether they modify an object assignment, a plan, a parameter, or the model itself. This checklist is an analytical device for the review, not an implemented standard or a formally verified contract system.

Table 2 Five fields for inspecting a language to symbolic interface

Field	What to record	Example	What the field does not establish
Representation	Exact emitted artifact and supported fragment	PDDL goal; Python program; grounded skill call	Semantic fidelity to the instruction
Binding	Entity identifiers, arguments, and evidence source	Perception-derived mug ID; supplied object inventory	Correct identity under ambiguity or change
Preconditions	Source and semantics of applicability information	Expert action model; assertion; learned value	Physical feasibility outside those assumptions
Validation	Checker, checked property, timing, and rejection rule	Parser; model-based check; motion planner	Properties outside the checker's jurisdiction
Feedback	Observation source and artifact changed by repair	Rebind entity; revise parameter; patch domain	Correctness of an unsupported patch

These fields are descriptive coding guidance proposed by this review. They are not a validated scoring scale.

A representation taxonomy alone hides substantial differences. Two systems can emit PDDL while one translates only a goal into a fixed domain and another generates the entire domain, including its preconditions. Their modeling burden and error opportunities differ. Two systems can emit Python while one executes a predetermined list and another repeatedly queries perception inside a feedback loop. The latter distinction concerns the program's observation semantics, not its file extension. Similarly, learned skill values and explicit logical preconditions provide different evidence for action acceptance.

Figure 1 shows five distinct obligations along the interface. Specification fidelity asks whether the formal task preserves the user's intended constraints. Binding correctness asks whether symbolic arguments denote the intended scene entities. Model-relative validity asks whether actions satisfy the encoded transition rules and goals. Physical feasibility asks whether the proposed actions admit execution under the robot and environment constraints. Execution confirmation asks whether the observed world exhibits the intended effect. These obligations interact but are not interchangeable. A checker can be exact with respect to an incomplete model, and a correct binding at planning time can become stale before execution.

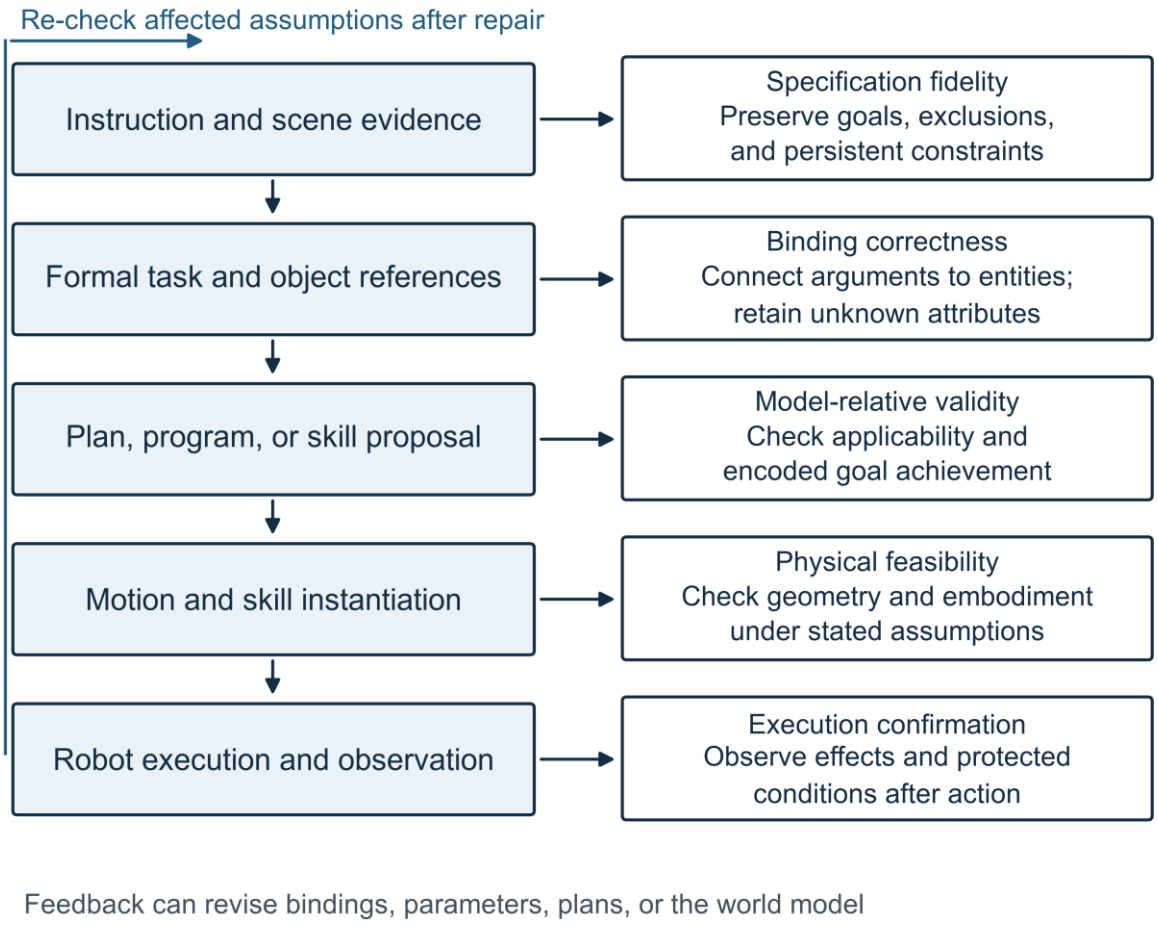


Fig. 1 Five checking obligations along a language to symbolic robot task interface. Left boxes name the artifacts or stages; right boxes identify the corresponding obligation. Feedback can change an earlier artifact and requires affected assumptions to be checked again. The diagram is a conceptual synthesis and contains no measured results

3.2 A worked counterexample

Consider the instruction, "Put the clean red mug inside the cupboard and leave the blue mug untouched." Assume two red mugs are present, cleanliness is unobserved, and the cupboard is closed. This is an illustrative counterexample constructed for analysis, not a benchmark episode or a reported experiment. A translation that requires only inside(red_mug_1, cupboard) can be syntactically correct and solvable while omitting

cleanliness and the protection of the blue mug. A plan validator cannot recover those omitted constraints from the instruction if it receives only that formalization.

The same example exposes errors beyond translation. Binding red_mug_1 to the dirty mug can satisfy every predicate in an incorrectly populated initial state. A plan that opens the cupboard before placing the mug may be logically applicable but geometrically infeasible because the door's swept volume is obstructed. An executor may report a successful placement call even if the mug slips after release. Figure 2 separates these failures by the evidence needed to detect them. The point is not that every system needs one universal validator, but that each acceptance decision should name the property it actually checks.

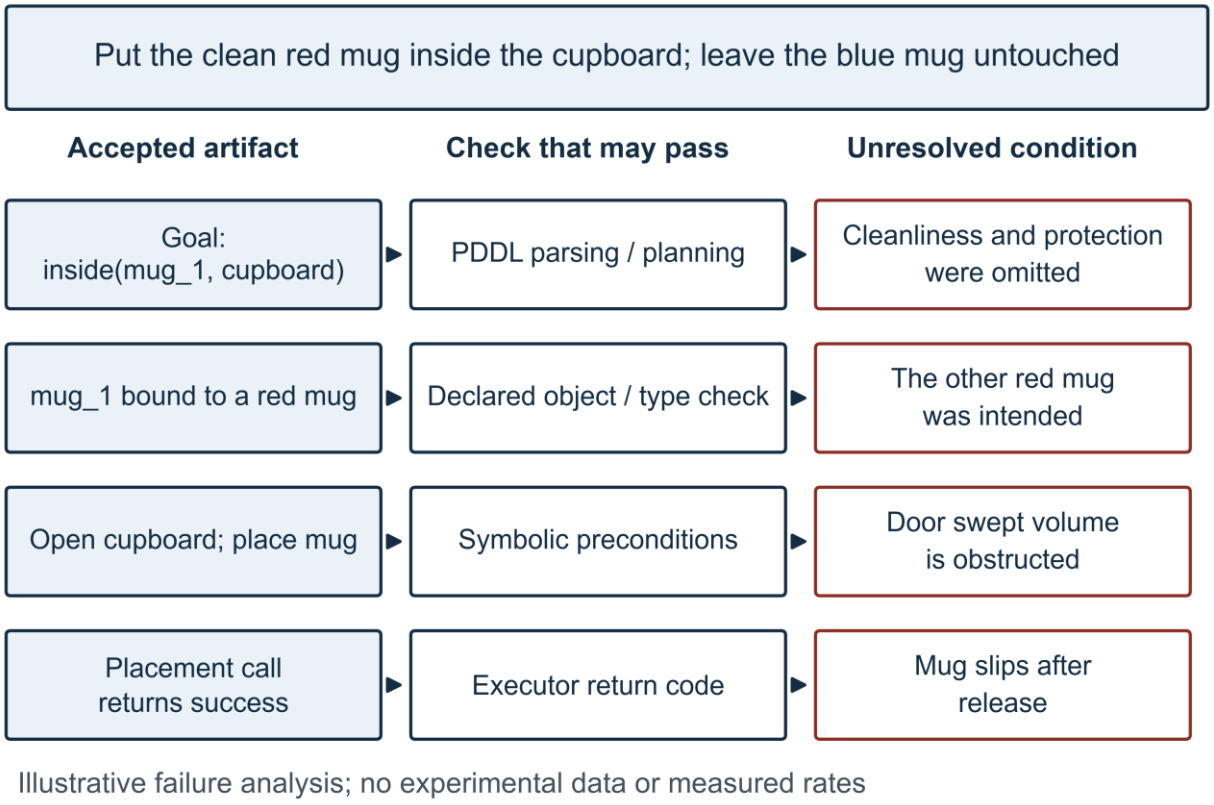


Fig. 2 Distinct failures in the illustrative mug instruction. Each row pairs an accepted artifact with a check that may pass and a condition that remains unresolved. The rows illustrate different failure possibilities rather than a measured sequence or an experiment

3.3 Preconditions, affordances, and unknown information

A symbolic precondition is a condition in an action model. A learned affordance or value estimate predicts the usefulness or feasibility of a skill under its training and observation conditions. A runtime assertion tests a condition available through a program interface. These mechanisms may complement each other, but a high value estimate does not logically establish a precondition. An assertion may be operationally helpful while relying on a detector whose error rate is unmeasured. Calling all three mechanisms "grounding" obscures the different claims they support.

Unknown information deserves a separate treatment. In a conventional closed-world encoding, an absent fact is commonly treated as false. A robot may instead lack an

observation of whether a mug is clean. Converting absence of evidence into a negative predicate can make a valid task appear impossible; inventing a positive fact can make an unsafe action appear applicable. An interface should distinguish observed facts, inferred facts, and unresolved conditions. This is a design recommendation developed from the comparison, not a capability uniformly implemented in the reviewed systems.

4 Compiling language into goals and world models

4.1 Goal translation under a fixed domain

The smallest compilation interface preserves the action model and translates the user's task into a formal goal, sometimes also constructing the initial problem instance. Xie and colleagues' public preprint directly investigates language-to-goal translation and documents sensitivity to spatial, numerical, and prompt conditions [12]. LLM+P similarly places a language interface in front of a classical planner while assuming an available PDDL domain [13]. These are valuable architectural reference points, although their preprint status and benchmark conditions should remain visible.

The central benefit is that search over action consequences can be delegated to a symbolic planner. The central limitation is that search begins after interpretation. If the translator selects the wrong object, weakens a quantifier, omits a persistence condition, or invents an initial fact, solving the resulting problem does not establish fidelity to the request. A planning failure is also ambiguous: the task may be impossible, the domain may be incomplete, or the translation may be wrong. Consequently, "planner success" is an insufficient stand-alone measure of language understanding.

Recent benchmarks make this distinction more explicit. Planetarium evaluates natural-language-to-PDDL problem generation using an equivalence-based procedure instead of relying only on surface agreement [14]. Its scope concerns particular classical planning domains and supported forms of equivalence, rather than unrestricted semantic equivalence for arbitrary robot models. Kagitha and colleagues examine generation pipelines through intermediate representations, including Python-based representations of PDDL [6]. These studies support separating formalization from solution search. They do not remove the need to evaluate grounding against a robot's observations.

4.2 Generating and revising the domain

When actions and predicates are also generated, interpretation extends into the robot's world model. Guan and colleagues construct PDDL domains with LLM assistance and corrective feedback from validators and human experts [15]. The distinction between these feedback sources is consequential: syntax checking can expose malformed expressions, whereas an expert can identify missing effects or factual errors. The public NL2Plan manuscript decomposes modeling into stages that produce types, actions, predicates, and a problem before invoking a planner [16]. Its inspectable intermediate artifacts make the interface's assumptions accessible, but do not independently establish that the resulting model matches the world.

Model generation changes what a successful plan means. If the same generation process invents an action and omits its difficult precondition, subsequent planning may become easier for the wrong reason. A validator using that domain can faithfully accept the

resulting shortcut. Domain acquisition should therefore be evaluated against transition evidence or expert specifications that are independent of the generated model. Testing only whether the generated domain admits a plan risks rewarding permissive models. This is a construct-validity concern, not a criticism that applies only to LLM-based acquisition.

InterPreT provides a different connection between language and model construction [17]. Human feedback during interaction helps generate executable predicate functions, which support learning operators and compiling a PDDL model. The important interface is the path from a linguistic condition to an observable predicate, not simply text generation of a domain file. The approach also makes the dependence on human feedback and accessible state features part of the evidence boundary. Predicate learning cannot be assumed to resolve perceptual uncertainty that the available sensing interface never exposes.

4.3 Scaling the problem without losing its meaning

Scene graphs organize relevant entities and relationships before formalization or action generation. DELTA uses such structure for problem construction and goal decomposition, reducing the burden placed on planning a large task at once [18]. SayPlan combines hierarchical scene-graph search, navigation planning, and simulated checking of candidate task plans [19]. These systems address a practical bottleneck: a language interface cannot consume every detail of a large environment indefinitely. Their mechanisms are related, but pruning a problem and validating an action sequence are different operations.

An abstraction must preserve the information needed for the requested task. An object irrelevant to the goal may still block a motion, hold a required tool, or violate an instruction to leave something unchanged. A sequence of locally achievable subgoals can also interfere through shared resources or delete effects. The useful question is therefore which constraints survive retrieval, pruning, and decomposition. Evidence for planning efficiency should be accompanied by evidence that the retained problem preserves the original goal and necessary dependencies. Table 3 compares representative compilation and model-repair interfaces without ranking their reported success rates.

Table 3 Representative compilation and model-repair interfaces

System and output	Binding and model source	Checking or repair	Evidence boundary
Guan et al. [15] PDDL domain	Predicates and actions generated; expert feedback	Validator diagnostics and human model correction	Solvability alone does not validate learned effects
NL2Plan [16] PDDL domain/problem	Staged extraction from text	Feedback on intermediate modeling artifacts	Public/workshop manuscript; not a journal result
InterPreT [17] Predicates Operators	Language feedback and executable predicate functions	Predicate correction; operator learning	Needs feedback and accessible state features

System and output	Binding and model source	Checking or repair	Evidence boundary
DELTA [18] PDDL problems Subgoals	Pruned scene graph; supplied planning domain	Formal planning after goal decomposition	Pruning must preserve relevant dependencies
SayPlan [19] Task actions	Hierarchical 3D scene graph	Scene-graph simulation; iterative replanning	Simulator acceptance differs from physical completion
LASP [20] Updated planning model	Incomplete preconditions, properties, objects	Execution-error-driven model refinement	New properties still require grounded evidence
AutoGPT+P [21] Affordance-based planning inputs	Object detection and affordance mapping	Planning/tool feedback; missing-information handling	Generated affordances can be mistaken

Descriptions refer to representative configurations in the sources. No success rates are pooled or ranked.

5 Generating actions, programs, and skill calls

5.1 Action sequences with environment feedback

An action-list interface asks the language model to choose a succession of task actions. Its simplicity makes generated behavior easy to inspect, but it also transfers ordering and state-tracking responsibilities to the model. LLM-Planner uses grounded replanning for embodied agents and evaluates the resulting system in ALFRED [22]. Inner Monologue continually incorporates textual feedback such as scene descriptions and success detection [23]. Both illustrate why a task interface should expose the information available after each action, rather than being described only as an LLM-generated plan.

Feedback quality constrains what replanning can achieve. A correct textual observation can reveal a failed step, but it may not expose the hidden cause. A generic failure message leaves object binding, unmet preconditions, and geometric blockage entangled. Repetition can then consume the retry budget without changing the relevant assumption. Conversely, a rich simulator message can make the interface appear more capable than it would be with noisy sensing. Comparisons should state whether feedback comes from ground-truth state, an engineered detector, learned perception, or a person.

State persistence also matters. Statler separates maintaining a world-state estimate from selecting subsequent behavior [24]. This design makes state tracking explicit and inspectable across interactions. An explicit memory, however, can preserve an incorrect belief as effectively as a correct one. Its reliability depends on how observations revise stored facts and how contradictions are handled. Memory organization is consequently a

distinct interface choice from access to new evidence, even when both improve performance in the evaluated tasks.

5.2 Programs as task interfaces

Programs support loops, conditionals, geometry calculations, and repeated perception calls that an action list may express awkwardly. Code as Policies generates programs that compose perception and control application programming interfaces (APIs) [25]. ProgPrompt provides available objects and actions through a program-oriented prompt and uses assertions with recovery behavior [26]. These approaches constrain generation through a robot's callable vocabulary while allowing flexible composition. The vocabulary's existence still does not guarantee that every generated call has the right arguments or that every condition is checked.

Program correctness should be decomposed into several properties. Parsing and type checks can reject malformed or incompatible calls. Symbol resolution can detect an undeclared object name. Assertions can expose a failed condition if a suitable observation function exists. None of these alone proves that a loop will terminate, that a sequence preserves all task constraints, or that a perception query refers to the intended entity. Program expressiveness creates opportunities for better control flow alongside additional obligations concerning termination, exception handling, and state updates.

Code generation and symbolic planning need not be competing approaches. A program can implement predicate grounding, call a planner, and monitor execution; a planner can select symbolic actions that invoke generated or fixed programs. InterPreT illustrates this connection through executable predicate functions and learned operators [17]. The boundary to inspect is the semantics of the generated function: what state it reads, what condition it claims to recognize, and what evidence was used to correct it. This is more informative than deciding whether a system belongs exclusively to a "code" or "PDDL" category.

5.3 Selecting skills and estimating feasibility

SayCan combines language relevance with learned value functions for available skills [27]. Its interface constrains action selection to a skill repertoire and introduces a state-dependent feasibility signal. This differs from asking a planner to prove applicability under a discrete action model. A value function can encode aspects of embodiment that a hand-written predicate model misses, but its interpretation depends on skill training, calibration, and the current observation distribution. It should not be described as a formal safety certificate.

Text2Motion combines language guidance, skill feasibility information, symbolic goals, and geometric reasoning to construct task and motion plans [28]. LLM3 proposes symbolic action sequences and continuous parameters, then uses motion-planning failure feedback to revise proposals [29]. These methods expose a continuum between selecting a known skill and proposing the parameters required to instantiate it. A symbolic action can remain plausible while its parameter assignment fails, making it necessary to distinguish task-level rejection from motion-level rejection.

VoxPoser is a useful boundary case [30]. Generated code constructs spatial value maps that guide trajectory optimization rather than merely naming a discrete skill. Its inclusion demonstrates that a symbolic language interface can produce objectives for a continuous planner. AutoTAMP similarly translates language into signal temporal logic (STL) specifications and uses checking and re-prompting around a TAMP solver [31]. STL is not PDDL, and a temporal specification is not an action sequence. These distinctions matter when selecting metrics and interpreting a system's guarantees. Table 4 summarizes representative action, program, and skill interfaces.

Table 4 Representative action, program, and skill interfaces

System and artifact	Grounding mechanism	Check or feedback	Main limitation of inference
LLM-Planner [22] Action list	Observed entities and low-level controller	Grounded replanning	ALFRED evidence is simulation evidence
Inner Monologue [23] Skill sequence	Textual scene/success information	Feedback-conditioned planning	Detection quality determines visible failures
Code as Policies [25] Program	Perception/control APIs	Program logic and API execution	Compilation is not full task verification
ProgPrompt [26] Program	Objects and action imports	Assertions and recovery routines	Checks depend on available state feedback
SayCan [27] Skill choice	Language relevance and skill values	Learned state-dependent feasibility	A value estimate is not a logical certificate
Text2Motion [28] Task/motion plan	Skill feasibility and inferred goals	Planning with geometric reasoning	Skill/model assumptions bound feasibility
LLM3 [29] Actions and parameters	Symbolic sequence and continuous samples	Motion-failure reasoning	Quantitative evidence centers on box packing
VoxPoser [30] Spatial objectives	Code-built 3D value maps	Closed-loop trajectory optimization	Perception/dynamics remain conditional
AutoTAMP [31] STL specification	Language to temporal constraints	Syntax and semantic re-prompting	STL validity differs from object/task fidelity

Program, action, and specification interfaces can coexist. A paper's inclusion here does not imply exclusive membership in one family.

6 Validation and repair across representations

6.1 Validation authority and timing

The LLM-Modulo position advocates combining language-model proposals with external model-based critics [32]. For robot task interfaces, the important consequence is to name both the critic and its jurisdiction. A parser checks a language grammar. A symbolic checker tests applicability or goal achievement under a model. A motion planner tests geometric feasibility under its scene and dynamics assumptions. An execution monitor checks an observation-derived condition. An LLM explanation of why a proposal is correct is another proposal unless it is anchored to an independent test.

Checks can occur before planning, before an action is dispatched, during motion, or after a skill finishes. Their timing affects which errors remain reversible. A post-action monitor may detect that a protected object moved, but some consequences may be irreversible. A precondition check made against stale state may pass before another actor changes the scene. A useful interface therefore records not only the predicate value, but the observation or model version on which it depends. This timestamp and provenance recommendation follows from the mismatch between discrete planning and changing environments; it has not been validated as a universal design solution here.

Recent systems illustrate different placements of symbolic oversight. SCLPlan checks proposed actions through precondition verification and incorporates a global symbolic planner [33]. VOLTS places symbolic applicability checking within action-token generation [34]. The latter uses tokens representing grounded actions in benchmark domains, which limits the inference that can be drawn about unseen robot objects or continuous execution. These mechanisms show that checks can be integrated at different points in generation. Their reported results do not establish a representation-independent ordering of reliability.

6.2 Repairing the right artifact

A failed action can call for revising a parameter, choosing another action, rebinding an object, acquiring a missing observation, or correcting a domain model. LASP explicitly addresses incomplete knowledge of preconditions, objects, and properties in open-world task planning [20]. AutoGPT+P derives planning information from an affordance-oriented scene representation and supports interaction when information or objects are missing [21]. Their contributions are particularly relevant because repair may change the problem being solved, rather than simply request another sequence under the same assumptions.

These repairs have different evidential costs. Changing a grasp pose may preserve the task but requires a new feasibility check. Substituting another object can alter the user's goal and may need clarification. Adding a precondition narrows the action model; deleting one can create an unjustified shortcut. A repair trace should identify the failed check, the changed artifact, the evidence for the change, and the remaining unresolved conditions. Logging only the final successful plan can conceal extensive intervention or model revision.

The dynamic planner LLM-DP maintains observed facts and beliefs while using symbolic planning in a text-based embodied environment [35]. Its assumptions include perfect textual observations and successful execution of valid actions. This makes it informative about belief and planning interfaces, while bounding its relevance to noisy robots. A 2026 public preprint on planner-in-the-loop PDDL formalization provides a further example of separating parseability, solvability, and outcome-oriented checks [36]. Its role here is contextual; it is not treated as verified robot deployment evidence.

6.3 Asking for help and stopping

Clarification is a legitimate interface output when an instruction or binding remains ambiguous. KnowNo uses conformal prediction to align language-planner uncertainty and requests human help when needed [37]. Its assumptions about grounding, executable actions, calibration, and helpful human responses bound the associated assurances. The appropriate lesson is that an interface can expose uncertainty and defer a decision, not that human assistance provides an unconditional guarantee of successful robot execution.

Stopping deserves equal attention. A generated "done" token, a planner reaching a symbolic goal, a successful skill return, and a physically verified task outcome are distinct events. A completion test should inspect the goal and any persistent constraints that were translated from language. If the relevant property is not observable, the system should represent that uncertainty rather than silently treat a lack of error as success. This recommendation follows directly from the worked example and the differing validation mechanisms in the literature.

7 Evaluating an interface without conflating outcomes

7.1 Separate the units of success

Planning benchmarks isolate important capabilities, but their units of success differ. PlanBench tests planning and reasoning about state changes in formally specified domains [38]. AutoPlanBench automates conversion from PDDL into natural-language prompts for action-choice evaluation [39]. That direction is the reverse of language-to-PDDL problem generation in Planetarium [14]. Treating all three as interchangeable evidence for language understanding would obscure what is held fixed and what is generated. They are complementary tools for testing distinct interface components.

For an instruction-to-plan pipeline, four complementary outcome groups should be reported separately. Specification fidelity concerns the preservation of objects, relations, quantifiers, ordering, exclusions, and persistence requirements. Object-binding accuracy should be reported within this group and traced through execution. Symbolic validity concerns plan applicability and goal satisfaction in the declared model. Physical feasibility concerns whether the selected actions and parameters admit execution under the robot's constraints. Observed completion concerns what actually happened, including protected conditions and delayed failures. These outcomes can be evaluated separately. Their relationship does not imply statistical independence or an automatic implication between adjacent stages.

An interface should also account for rejected, unparseable, unresolved, and abandoned cases. Reporting task success only among successfully parsed outputs can conceal a

substantial formalization failure rate. Reporting validity only for returned plans can conceal frequent abstention. A complete account therefore needs a declared task denominator and separate coverage, rejection, clarification, and completion outcomes. A request for help is not automatically a task failure, but its frequency and cost should be visible alongside autonomous completion.

7.2 A matched comparison protocol

The proposed protocol holds the instruction set, object inventory, perception stream, skill library, language model, and total interaction budget fixed. It compares alternative interfaces over the same task instances and records every translation, checker call, intervention, and execution attempt. Where a representation requires extra modeling effort, that effort is reported rather than hidden in the setup. Fixed-domain and generated-domain conditions should be separate, because allowing a system to change its own transition rules changes the comparison's meaning.

Task variations should target the interface obligations. Object renaming and duplicate descriptions test identity resolution. Unobserved attributes test the treatment of unknown facts. Omitted or incorrect preconditions test model repair. Conjunctive goals with protected objects test preservation under decomposition. Geometric obstruction tests whether symbolic applicability is mistaken for physical feasibility. Delayed or contradictory observations test monitoring and state revision. These are proposed tests, not a dataset constructed or evaluated for this review. Table 5 maps each obligation to an outcome and a failure probe.

Table 5 Proposed evaluation outcomes and diagnostic probes

Obligation	Outcome to report	Diagnostic probe	Evidence needed
Specification fidelity	Constraints preserved, omitted, or altered	Negation; quantifiers; protected object	Independent intended specification or adjudication
Object binding	Correct IDs; ambiguity; unresolved attributes	Duplicate labels; renaming; hidden cleanliness	Entity-level annotations and observation provenance
Symbolic validity	Applicability and goal reachability	False or missing precondition	Declared fixed domain and validated trace
Physical feasibility	Feasible motion/skill instantiation	Obstructed door; unreachable target	Robot constraints and motion/execution checks
Observed completion	Goal and persistent constraints satisfied	Slip after release; protected-object movement	Post-action observations with stated accuracy

Obligation	Outcome to report	Diagnostic probe	Evidence needed
Repair and coverage	Recovery, rejection, help, and budget use	Stale state; contradictory feedback	Complete traces and all-task denominator

This is a proposed protocol, not an evaluated benchmark. Controlled ablations should hold the source of evidence fixed.

A useful diagnostic comparison replaces one source of interface information at a time. Oracle object identities isolate planning from binding; a fixed expert domain isolates formalization from model acquisition; deterministic execution isolates planning from skill reliability. These conditions should be labeled as diagnostic rather than deployed performance. End-to-end robot trials remain necessary for claims about observed physical completion. For repeated stochastic runs, studies should provide trial counts and uncertainty estimates appropriate to the sampling design, together with the model version, date, prompts, and retry policy.

Cost should include wall-clock planning time, checker and motion-planner calls, language-model calls or tokens, human interventions, and execution time. These quantities should not be collapsed into one score unless the weighting reflects a declared application. A slower interface may reject more unsafe or ambiguous proposals, while a faster one may shift work to manual model construction. Without observing these tradeoffs, a high task-success percentage gives too little information for choosing a robot architecture.

8 Discussion and research priorities

8.1 What the comparison supports

The comparison supports a bounded conclusion: explicit task interfaces make particular assumptions inspectable and particular errors rejectable. They do not, by themselves, make every assumption true. Fixed-domain compilation concentrates uncertainty in task specification and state construction. Domain generation adds uncertainty about transition rules. Program generation adds control-flow and API obligations. Skill selection adds dependence on the learned feasibility signal. Feedback mechanisms then determine which of these assumptions can change after a failure. This distribution of responsibility explains why a single label such as "neuro-symbolic planning" is insufficient for comparing systems.

This interpretation connects earlier work on language grounding with contemporary formalization and verification pipelines [2, 6, 32]. Its practical contribution is the alignment between an interface field, an acceptance test, and an evidence boundary. The tables are intended as design and reporting aids. They are not measurements of how often failures occur, a validated predictor of deployment reliability, or evidence that one output language dominates the alternatives. Such claims would require a larger, systematically sampled corpus or new matched experiments.

Several rival explanations remain when interpreting reported improvements. A stronger language model, a better skill library, richer feedback, a smaller object inventory, or more retries can improve results without changing the interface mechanism. Conversely, a restrictive interface may reduce errors partly by reducing the set of expressible tasks.

These alternatives make it necessary to report task coverage and the information supplied to each component. Interface constraints and language-model capability should be treated as separate experimental factors where feasible.

8.2 Binding should remain inspectable through execution

A useful next step is to carry object identity, perceptual evidence, and unresolved attributes with symbolic references. The technical question is whether this information enables a checker to detect incorrect or stale bindings before action dispatch. An informative comparison would hold perception fixed and vary whether the executor receives only an object name or an identity record with evidence and update rules. It should measure binding errors, unnecessary clarification, and physical task completion rather than textual agreement alone.

The corresponding challenge is maintaining useful abstractions. A large language model should not need every point-cloud sample to plan a household task, yet a compact symbolic description should not discard an obstacle or a protected relation. Retrieval and decomposition therefore require tests of constraint preservation, not only token reduction. Scene-graph systems provide concrete starting points [18, 19], but their abstractions should be stress-tested with task-irrelevant entities that remain physically or causally relevant. Such tests would clarify when abstraction helps planning and when it changes the problem.

8.3 Model repair needs evidence beyond self-consistency

Domain generation and repair are promising where manual modeling is expensive, but a repaired model can become more internally consistent without becoming more accurate. Transition observations, independently specified constraints, and explicit human corrections provide different forms of evidence. Future work should distinguish model patches supported by observations from patches proposed only to make a plan solvable. Regression tests on previously validated transitions could then expose repairs that fix one task while breaking another.

Feedback also needs a causal vocabulary. "Cannot execute" is a useful alert, but a failure trace that identifies an unbound object, a false precondition, a collision, or a missing observation supports a more targeted intervention. The testable question is whether structured failure reports reduce repeated errors under the same total budget. A matched study should compare the report format while preserving the underlying checker, rather than simultaneously upgrading both the feedback generator and the planner.

8.4 Limits of this review

The review's targeted search and purposive selection can omit relevant systems and overrepresent well-documented, accessible projects. Source-level mechanisms were checked through targeted reading, but the implementations and performance claims were not independently reproduced. Some classifications depend on representative configurations in papers that support several variants. They should be updated if later versions change their interfaces. The rapidly changing model ecosystem also limits the transfer of a reported performance result to a newer model or a different robot.

The checklist itself is a synthesis proposed for inspection and evaluation. Its usefulness has not been tested through independent coding agreement or prospective architecture studies. Formal models with partial observability, rich temporal constraints, multi-robot coordination, and end-to-end learned policies extend beyond the main comparison. Consequently, the recommended protocol should be adapted to those settings rather than assumed sufficient. The durable contribution is the distinction between what an interface represents, what its checker establishes, and what execution actually confirms.

9 Conclusion

Language to symbolic robot task interfaces differ in where they place responsibility for objects, action conditions, and feedback. Comparing only their output representation conceals these differences. This review connects goal and model compilation, action and program generation, and skill selection through a common checklist of binding, preconditions, validation authority, and repair. The resulting comparison separates specification fidelity, model-relative validity, physical feasibility, and observed completion. Future evaluations can use these distinctions to report what has been checked and under which assumptions. The reviewed evidence supports explicit, bounded claims about interface behavior; it does not justify equating a valid symbolic artifact with dependable physical execution.

Declarations

Funding

[AUTHOR TO COMPLETE: funding sources and grant identifiers, or an accurate statement of no specific funding.]

Competing interests

[AUTHOR TO COMPLETE: all relevant financial and non-financial interests, or an accurate declaration of none.]

Author contributions

[AUTHOR TO COMPLETE: verified contributions of every named author.]

Data availability

No new robot experiments or performance datasets were generated. The extracted literature evidence and search record are provided in Online Resources 1 and 2. Source publications are identified by persistent links in those files and in the references. The coding guidance and proposed evaluation protocol are supplied as Online Resource 3.

Ethics approval and consent

This literature review did not recruit participants, conduct animal studies, or collect new personal data. Ethics approval and participant consent are not applicable to the review activities described here.

Consent for publication

Not applicable to the review activities described here.

Supplementary information

Online Resource 1 Literature evidence ledger and bibliographic verification records, supplied as OnlineResource1_Evidence.csv. Online Resource 2 Exact discovery queries and targeted retrieval record, supplied as OnlineResource2_Searches.csv. Online Resource 3 Coding guidance, source-status rules, and the proposed matched evaluation protocol, supplied as OnlineResource3_Methods.pdf.

References

1. Tellex S, Gopalan N, Kress-Gazit H, Matuszek C. Robots That Use Language. Annual Review of Control, Robotics, and Autonomous Systems. 2020;3(1):25-55. https://doi.org/10.1146/annurev-control-101119-071628

2. Cohen V, Liu JX, Mooney R, Tellex S, Watkins D. A Survey of Robotic Language Grounding: Tradeoffs between Symbols and Embeddings. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence. 2024:7999-8009. https://doi.org/10.24963/ijcai.2024/885

3. Kim Y, Kim D, Choi J, Park J, Oh N, Park D. A survey on integration of large language models with intelligent robots. Intelligent Service Robotics. 2024;17(5):1091-1107. https://doi.org/10.1007/s11370-024-00550-5

4. Yao X, Zhou H, Mees O, Meng Y, Xiao T, Bisk Y, et al. Bridging language and action: A survey of language-conditioned robot manipulation. The International Journal of Robotics Research. 2026:02783649261468360. https://doi.org/10.1177/02783649261468360

5. Khan RU, Aljuaid H, Ning Z. A critical assessment of the rise of agentic AI, its capabilities, task domains and open challenges. Discover Artificial Intelligence. 2026;6(1):1343. https://doi.org/10.1007/s44163-026-02342-5

6. Kagitha PP, Sun B, Desai I, Zhu A, Huang C, Li M, et al. Unifying Inference-Time Planning Language Generation. Findings of the Association for Computational Linguistics: ACL 2026. 2026:8531-8574. https://doi.org/10.18653/v1/2026.findings-acl.415

7. Tan K. From Planner to Policy: A Survey of Large Language Models for Robot Control. Public project-page survey [Internet]. 2026. https://tantansir.github.io/llm-for-robot-control/ [cited 19 October 2026; paper listed as forthcoming].

8. Bian S, Zhang Y, Tian G, Miao Z, Wu EQ, Yang SX, et al. Large language model-based task planning for service robots: A review. Biomimetic Intelligence and Robotics. 2026;6(1):100274. https://doi.org/10.1016/j.birob.2026.100274

9. Fox M, Long D. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. Journal of Artificial Intelligence Research. 2003;20:61-124. https://doi.org/10.1613/jair.1129

10. Helmert M. The Fast Downward Planning System. Journal of Artificial Intelligence Research. 2006;26:191-246. https://doi.org/10.1613/jair.1705

11. Garrett CR, Chitnis R, Holladay R, Kim B, Silver T, Kaelbling LP, et al. Integrated Task and Motion Planning. Annual Review of Control, Robotics, and Autonomous Systems. 2021;4(1):265-293. https://doi.org/10.1146/annurev-control-091420-084139

12. Xie Y, Yu C, Zhu T, Bai J, Gong Z, Soh H. Translating Natural Language to Planning Goals with Large-Language Models. arXiv [Preprint]. 2023. https://doi.org/10.48550/arXiv.2302.05128

13. Liu B, Jiang Y, Zhang X, Liu Q, Zhang S, Biswas J, et al. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv [Preprint]. 2023. https://doi.org/10.48550/arXiv.2304.11477

14. Zuo M, Velez FP, Li X, Littman M, Bach S. Planetarium: A Rigorous Benchmark for Translating Text to Structured Planning Languages. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). 2025:11223-11240. https://doi.org/10.18653/v1/2025.naacl-long.560

15. Guan L, Valmeekam K, Sreedharan S, Kambhampati S. Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning. Advances in Neural Information Processing Systems 36. 2023:79081-79094. https://doi.org/10.52202/075280-3459

16. Gestrin E, Kuhlmann M, Seipp J. NL2Plan: Robust LLM-Driven Planning from Minimal Text Descriptions. arXiv [Preprint]. 2024 [revised 2025]. https://doi.org/10.48550/arXiv.2405.04215

17. Han M, Zhu Y, Zhu SC, Wu YN, Zhu Y. INTERPRET: Interactive Predicate Learning from Language Feedback for Generalizable Task Planning. Robotics: Science and Systems XX. 2024. https://doi.org/10.15607/rss.2024.xx.034

18. Liu Y, Palmieri L, Koch S, Georgievski I, Aiello M. DELTA: Decomposed Efficient Long-Term Robot Task Planning Using Large Language Models. 2025 IEEE International Conference on Robotics and Automation (ICRA). 2025:10995-11001. https://doi.org/10.1109/icra55743.2025.11127838

19. Rana K, Haviland J, Garg S, Abou-Chakra J, Reid I, Suenderhauf N. SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning. Conference on Robot Learning (Proceedings of Machine Learning Research). 2023;229:23-72. https://proceedings.mlr.press/v229/rana23a.html

20. Chen G, Yang L, Jia R, Hu Z, Chen Y, Zhang W, et al. Language-Augmented Symbolic Planner for Open-World Task Planning. Robotics: Science and Systems XX. 2024. https://doi.org/10.15607/rss.2024.xx.037

21. Birr T, Pohl C, Younes A, Asfour T. AutoGPT+P: Affordance-based Task Planning using Large Language Models. Robotics: Science and Systems XX. 2024. https://doi.org/10.15607/rss.2024.xx.112

22. Song CH, Wu J, Washington C, Sadler BM, Chao WL, Su Y. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. 2023 IEEE/CVF International Conference on Computer Vision (ICCV). 2023:2986-2997. https://doi.org/10.1109/iccv51070.2023.00280

23. Huang W, Xia F, Xiao T, Chan H, Liang J, Florence P, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. Conference on Robot Learning (Proceedings of Machine Learning Research). 2023;205:1769-1782. https://proceedings.mlr.press/v205/huang23c.html

24. Yoneda T, Fang J, Li P, Zhang H, Jiang T, Lin S, et al. Statler: State-Maintaining Language Models for Embodied Reasoning. 2024 IEEE International Conference on Robotics and Automation (ICRA). 2024:15083-15091. https://doi.org/10.1109/icra57147.2024.10610634

25. Liang J, Huang W, Xia F, Xu P, Hausman K, Ichter B, et al. Code as Policies: Language Model Programs for Embodied Control. 2023 IEEE International Conference on Robotics and Automation (ICRA). 2023:9493-9500. https://doi.org/10.1109/icra48891.2023.10160591

26. Singh I, Blukis V, Mousavian A, Goyal A, Xu D, Tremblay J, et al. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. 2023 IEEE International Conference on Robotics and Automation (ICRA). 2023:11523-11530. https://doi.org/10.1109/icra48891.2023.10161317

27. Ahn M, Brohan A, Brown N, Chebotar Y, Cortes O, David B, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. Conference on Robot Learning (Proceedings of Machine Learning Research). 2023;205:287-318. https://proceedings.mlr.press/v205/ichter23a.html

28. Lin K, Agia C, Migimatsu T, Pavone M, Bohg J. Text2Motion: from natural language instructions to feasible plans. Autonomous Robots. 2023;47(8):1345-1365. https://doi.org/10.1007/s10514-023-10131-7

29. Wang S, Han M, Jiao Z, Zhang Z, Wu YN, Zhu SC, et al. LLM3: Large Language Model-based Task and Motion Planning with Motion Failure Reasoning. 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2024:12086-12092. https://doi.org/10.1109/iros58592.2024.10801328

30. Huang W, Wang C, Zhang R, Li Y, Wu J, Fei-Fei L. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. Conference on Robot Learning (Proceedings of Machine Learning Research). 2023;229:540-562. https://proceedings.mlr.press/v229/huang23b.html

31. Chen Y, Arkin J, Dawson C, Zhang Y, Roy N, Fan C. AutoTAMP: Autoregressive Task and Motion Planning with LLMs as Translators and Checkers. 2024 IEEE International Conference on Robotics and Automation (ICRA). 2024:6695-6702. https://doi.org/10.1109/icra57147.2024.10611163

32. Kambhampati S, Valmeekam K, Guan L, Verma M, Stechly K, Bhambri S, et al. Position: LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks. International Conference on Machine Learning (Proceedings of Machine Learning Research). 2024;235:22895-22907. https://proceedings.mlr.press/v235/kambhampati24a.html

33. Byrd G, Rivera C, Kemp B, Booker M, Schmidt A, de Melo CM, et al. Constrained natural language action planning for resilient embodied systems. npj Artificial Intelligence. 2026;2(1):73. https://doi.org/10.1038/s44387-026-00117-y

34. Massad N, Trabelsi A, Ferland F, Kabanza F. VOLTS: Validated Output through Logit Tree Search for Reliable PDDL Planning with Small Language Models. The 39th Canadian Conference on Artificial Intelligence (Proceedings of Machine Learning Research). 2026;318:872-879. https://proceedings.mlr.press/v318/massad26a.html

35. Dagan G, Keller F, Lascarides A. Dynamic Planning with a LLM. arXiv [Preprint]. 2023. https://doi.org/10.48550/arXiv.2308.06391

36. Jiang J, Zhang J, Mo F, Li L, Zeng D. Toward Secure and Reliable PDDL Formalization of Large Language Models with Planner-in-the-Loop Feedback. arXiv [Preprint]. 2026. https://doi.org/10.48550/arXiv.2606.29700

37. Ren AZ, Dixit A, Bodrova A, Singh S, Tu S, Brown N, et al. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. Conference on Robot Learning (Proceedings of Machine Learning Research). 2023;229:661-682. https://proceedings.mlr.press/v229/ren23a.html

38. Valmeekam K, Marquez M, Olmo A, Sreedharan S, Kambhampati S. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. Advances in Neural Information Processing Systems 36. 2023:38975-38987. https://doi.org/10.52202/075280-1693

39. Stein K, Fišer D, Hoffmann J, Koller A. Automating the Generation of Prompts for LLM-based Action Choice in PDDL Planning. Proceedings of the International Conference on Automated Planning and Scheduling. 2025;35(1):250-259. https://doi.org/10.1609/icaps.v35i1.36126